

Infralytix

Test Plan & Test Cases

Testing strategy, environments, entry/exit criteria, and concrete test cases covering all ten Intelligence Modules — plus performance, security, and acceptance testing.

Unit → System → UAT

Security Test Cases

Performance Benchmarks

Defect Workflow

Table of Contents

Test Plan & Test Cases

1	Introduction & Objectives	03
	What this plan covers and why	
2	Test Strategy	04
	Levels of testing and approach at each level	
3	Test Environment & Tools	05
	Where and how tests run	
4	Entry & Exit Criteria	06
	When testing starts and when it's "done"	
5	Functional Test Cases	07
	Module-by-module, mapped to FR IDs	
6	Non-Functional Test Cases	10
	Performance, security, and reliability tests	
7	Defect Management	11
	Severity levels and resolution workflow	
8	User Acceptance Testing	12
	Sign-off checklist for "done"	

1

Introduction & Objectives

What this plan covers and why

This Test Plan defines how Infralytix will be verified against the requirements in the SRS (Document 2 of 5) and the design in the SDD (Document 3 of 5). It covers functional correctness for all ten Intelligence Modules, plus the non-functional qualities — performance, security, and reliability — called out as NFRs.

Objective	How this plan addresses it
Verify every Must/Should FR	Section 5 maps at least one test case to each FR ID from the SRS
Catch AI-output regressions	Golden-sample test repos/logs with expected diagnosis shape, re-run on every Agent change
Validate NFRs, not just features	Section 6 dedicates test cases to performance and security targets
Prevent "it works on my machine"	All tests run against the same Docker Compose environment used in the Deployment Guide

2

Test Strategy

Levels of testing and approach at each level

Level	Scope	Tooling	Owner / Cadence
Unit	Individual Agent logic, Pydantic schemas, service functions	pytest, pytest-mock	Every commit (CI)
Integration	API routes ↔ services ↔ database; Agent ↔ Gemini API (mocked)	pytest + httpx test client, testcontainers (Postgres)	Every PR (CI)
System (E2E)	Full user flows through the running frontend + backend stack	Playwright	Before each phase merge
Security	Auth, RBAC, secret-handling, injection resistance	Manual checklist + automated dependency/secret scans	Before each release
Performance	Response-time and concurrency targets from the SRS (NFR-P1-P4)	Locust / k6	Before each release
UAT	Persona-driven walkthroughs of representative use cases	Manual, against the Use Case Models in the SRS	Before v1.0 sign-off

AI OUTPUT TESTING APPROACH

Because Agent output is generative, functional tests validate *structure and required fields* (e.g. "root_cause, severity, and fix are all non-empty and severity ∈ {info, warning, error, critical}") rather than asserting exact wording — with a small set of golden-sample fixtures spot-checked for quality on every Agent prompt change.

3

Test Environment & Tools

Where and how tests run

Environment	Description
Local	Docker Compose stack (frontend, backend, PostgreSQL) — same as the Deployment Guide's local setup, with Gemini/GitHub calls mocked
CI	GitHub Actions runners; ephemeral Postgres via service containers; unit + integration suites on every push
Staging	Cloud-hosted mirror of production with real (rate-limited) external API access, used for E2E and performance runs

Tool	Purpose
pytest / pytest-asyncio	Backend unit & integration tests
httpx.AsyncClient	Exercising FastAPI routes in-process
Playwright	Browser-driven end-to-end scenarios
Locust / k6	Load and latency testing against staging
Bandit / pip-audit	Static security & dependency vulnerability scanning

4

Entry & Exit Criteria

When testing starts, and when it's "done"

Entry Criteria

- The relevant phase's modules are code-complete and pass their own unit test suite locally.
- A staging deployment is reachable, matching the Deployment Guide's production topology.

Exit Criteria

- 100% of Must-priority (P1) functional test cases pass; $\geq 90\%$ of Should-priority (P2) pass.
- No open Critical or High severity defects (see Section 7 for severity definitions).
- All NFR targets in Section 6 are met on staging.
- UAT checklist in Section 8 is fully signed off for the phase's scope.

KNOWN LIMITATION, ACCEPTED FOR V1.0

AI-generated content cannot be asserted byte-for-byte; exit criteria for FR test cases involving Gemini output are based on structural validation and a manual quality spot-check, not full determinism.

5

Functional Test Cases

Module-by-module, mapped to FR IDs from the SRS

Project Intelligence (FR1)

TC ID	FR	Description	Expected Result	Priority
TC-101	FR1.1	Upload a valid zipped Node.js repo	Project created; status = analyzed	P1
TC-102	FR1.2	Analyze a Python/FastAPI repo	Detected stack includes "Python", "FastAPI"	P1
TC-103	FR1.3	Analyze repo with an outdated dependency	Dependency analysis flags the outdated package	P2

Deployment Intelligence (FR2)

TC ID	FR	Description	Expected Result	Priority
TC-201	FR2.1	Generate Dockerfile for a detected Node.js + Express stack	Dockerfile builds successfully via <code>docker build</code>	P1
TC-202	FR2.2	Generate Compose file for a repo with an API + a database service	Compose file defines both services with a working network	P1
TC-203	FR2.3	Generate GitHub Actions workflow	Workflow YAML passes <code>actionlint</code> validation	P1

Log Intelligence (FR4)

TC ID	FR	Description	Expected Result	Priority
TC-401	FR4.1	Submit a Python <code>KeyError</code> traceback	Response includes <code>root_cause</code> referencing the missing key	P1
TC-402	FR4.2	Submit a log with an OOM-killed container event	Severity classified as "critical"	P1
TC-403	FR4.3	Submit an Nginx 502 log	Root cause + at least one proposed fix returned	P1

Cloud Intelligence (FR5)

TC ID	FR	Description	Expected Result	Priority
TC-501	FR5.1	Request estimate for a 2-vCPU / 4GB workload	Ranged estimate returned for AWS, Azure, and GCP	P1
TC-502	FR5.2	Request optimization suggestions for an oversized instance	At least one right-sizing suggestion returned	P2

Security Intelligence (FR6)

TC ID	FR	Description	Expected Result	Priority
TC-601	FR6.1	Scan a repo containing a hardcoded AWS key pattern	Finding reported; raw key value never appears in the API response	P1
TC-602	FR6.2	Scan a Dockerfile running as root with an exposed privileged port	Both issues flagged with severity $\geq$ medium	P1

AI Engineer / Orchestrator (FR10)

TC ID	FR	Description	Expected Result	Priority
TC-1001	FR10.1	Ask "Why is my deployment failing?" with a recent failed run in context	Orchestrator routes to Log Intelligence Agent; coherent answer returned	P1
TC-1002	FR10.3	Ask "Generate Terraform for this project"	Request recognized; Phase-4 scope message or generated output returned (per phase availability)	P3

6

Non-Functional Test Cases

Performance, security, and reliability tests

TC ID	NFR	Test	Pass Criteria
TC-P01	NFR-P1	Load-test Log Intelligence with 20 concurrent diagnosis requests	P95 response time < 30s
TC-P02	NFR-P2	Measure dashboard first-load time on a broadband connection	< 2.5s to interactive
TC-P03	NFR-P4	Run 10 concurrent analysis jobs (Project Intelligence)	No job errors or timeouts
TC-S01	NFR-S1/S2	Attempt API access with an expired JWT	401 Unauthorized returned
TC-S02	NFR-S1/S2	Attempt to access another user's project by guessing its ID	403 Forbidden returned (RBAC enforced)
TC-S03	Security design	Inspect logs and DB after a Security Intelligence scan finds a secret	No raw secret value present anywhere at rest
TC-R01	Reliability	Simulate a Gemini API timeout during Log Intelligence diagnosis	Graceful error returned; no 500 crash

7

Defect Management

Severity levels and resolution workflow

Severity	Definition	Response Target
Critical	Data loss, security exposure, or a core flow completely broken	Fix before any further merge
High	A Must-priority (P1) requirement fails or is badly degraded	Fix before phase exit
Medium	A Should-priority (P2) requirement fails; workaround exists	Fix before v1.0 release
Low	Cosmetic or Nice-to-have (P3) issue	Backlog

Defect lifecycle: **New** → **Triaged** → **In Progress** → **Fixed** → **Verified** → **Closed**, tracked in the same GitHub repository as the code, linked to the failing test case ID for traceability back to this document.

8

User Acceptance Testing

Sign-off checklist for "done"

✓	Acceptance Item	Traces To
☐	Upload a real side-project repo and receive a correct stack detection	UC — Upload project for analysis
☐	Paste a genuine failure log and receive a useful root-cause explanation	UC-04 — Explain a failing deployment
☐	Generate a Dockerfile + Compose file that builds and runs without manual edits	UC — Generate deployment configuration
☐	Receive a cloud cost estimate that matches actual billing within $\pm 15\%$	UC — Estimate cloud cost
☐	Run a Security Intelligence scan on a repo with a planted secret and confirm it's caught	UC — Scan repo for exposed secrets
☐	View an auto-generated architecture diagram that matches the real project structure	UC — View architecture diagram
☐	Ask the AI Engineer an open-ended question and get a correctly-routed, coherent answer	FR10 / Orchestrator routing

NEXT DOCUMENT IN THIS SUITE

Document 5 — Deployment & Operations Guide: installation, Docker, CI/CD, monitoring, backups, and production deployment steps.